Operating System Practical Question Bank

Mock Test Preparation – Questions, Programs & Expected Outputs

1. Linux Environment and File Management

Q1. Write Linux commands to create a directory, create a file, and display the contents of the directory.

Answer:

```
mkdir OS_Lab
cd OS_Lab
touch sample.txt
ls
```

Expected Output:

```
sample.txt
```

Q2. Write Linux commands to create a file and display its contents.

Answer:

```
touch student.txt
echo "Operating System Lab" > student.txt
cat student.txt
```

Expected Output:

```
Operating System Lab
```

Q3. Write Linux commands to copy, move, rename, and delete a file.

Answer:

```
cp student.txt copy.txt
mv copy.txt newfile.txt
mv newfile.txt renamed.txt
rm renamed.txt
```

Expected Output:

```
File operations completed successfully.
```

Q4. Write a Linux command to change file permissions.

Answer:

```
chmod 755 sample.txt
ls -l sample.txt
```

Expected Output:

```
-rwxr-xr-x sample.txt
```

Q5. Write Linux commands to monitor running processes.

Answer:

```
ps
ps aux
top
```

Expected Output:

```
PID   USER   %CPU   %MEM   COMMAND
```

2. Shell Scripting and System Automation

Q6. Write a shell script using an if-else statement to check whether a number is positive or negative.

Answer:

```
#!/bin/bash
echo "Enter a number:"
read n

if [ $n -ge 0 ]
then
    echo "Positive Number"
else
    echo "Negative Number"
fi
```

Expected Output:

```
Enter a number:
25
Positive Number
```

Q7. Write a shell script to check whether a number is even or odd.

Answer:

```
#!/bin/bash
echo "Enter a number:"
read n

if [ $((n % 2)) -eq 0 ]
then
    echo "Even Number"
else
    echo "Odd Number"
fi
```

Expected Output:

```
Enter a number:
10
Even Number
```

Q8. Write a shell script using a case statement to perform basic arithmetic operations.

Answer:

```
#!/bin/bash
echo "Enter first number:"
read a
echo "Enter second number:"
read b

echo "1. Addition"
echo "2. Subtraction"
echo "3. Multiplication"
echo "4. Division"
echo "Enter your choice:"
read ch

case $ch in
1) echo "Result = $((a+b))" ;;
2) echo "Result = $((a-b))" ;;
3) echo "Result = $((a*b))" ;;
4) echo "Result = $((a/b))" ;;
*) echo "Invalid Choice" ;;
esac
```

Expected Output:

```
Enter first number:
10
Enter second number:
5
Enter your choice:
1
Result = 15
```

Q9. Write a shell script using a for loop to display numbers from 1 to 10.

Answer:

```
#!/bin/bash
for ((i=1; i<=10; i++))
do
    echo $i
done
```

Expected Output:

```
1
2
3
4
5
6
7
8
9
10
```

Q10. Write a shell script to calculate the factorial of a number.

Answer:

```
#!/bin/bash
echo "Enter a number:"
read n
fact=1

for ((i=1; i<=n; i++))
do
    fact=$((fact*i))
done

echo "Factorial = $fact
```

Expected Output:

```
Enter a number:
5
Factorial = 120
```

Q11. Write a shell script using a function to find the square of a number.

Answer:

```
#!/bin/bash
square()
{
    echo "Square = $(( $1 * $1 ))"
}

echo "Enter a number:"
read n
square $n
```

Expected Output:

```
Enter a number:
6
Square = 36
```

Q12. Write a shell script to check whether a file exists.

Answer:

```
#!/bin/bash
echo "Enter filename:"
read file

if [ -f "$file" ]
then
    echo "File exists"
else
    echo "File does not exist"
fi
```

Expected Output:

```
Enter filename:
student.txt
File exists
```

3. Process Creation, Scheduling and Performance Analysis

Q13. Write a C program to create a child process using fork() and display the process IDs.

Answer:

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid = fork();

    if (pid < 0)
        printf("Process creation failed\n");
    else if (pid == 0)
    {
        printf("Child Process\n");
        printf("Child PID = %d\n", getpid());
    }
    else
    {
        printf("Parent Process\n");
        printf("Parent PID = %d\n", getpid());
    }

    return 0;
}
```

Expected Output:

```
Parent Process
Parent PID = 1234
Child Process
Child PID = 1235
```

Q14. Write a C program to demonstrate parent and child process using wait().

Answer:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid = fork();

    if (pid == 0)
        printf("Child process is running\n");
    else if (pid > 0)
    {
        wait(NULL);
        printf("Parent process resumes after child\n");
    }
    else
        printf("Fork failed\n");

    return 0;
}
```

Expected Output:

```
Child process is running
Parent process resumes after child
```

Q15. Write a C program to implement FCFS CPU scheduling.

Answer:

```
#include <stdio.h>

int main()
{
    int n, i;
    int bt[20], wt[20], tat[20];
    float avgwt = 0, avgtat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        printf("Enter burst time of P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    wt[0] = 0;
    for(i = 1; i < n; i++)
        wt[i] = wt[i-1] + bt[i-1];

    for(i = 0; i < n; i++)
    {
        tat[i] = wt[i] + bt[i];
        avgwt += wt[i];
        avgtat += tat[i];
    }

    printf("\nProcess\tBT\tWT\tTAT\n");
    for(i = 0; i < n; i++)
        printf("P%d\t%d\t%d\t%d\n", i+1, bt[i], wt[i], tat[i]);

    printf("\nAverage Waiting Time = %.2f", avgwt/n);
    printf("\nAverage Turnaround Time = %.2f\n", avgtat/n);

    return 0;
}
```

Expected Output:

```
Enter number of processes: 3
Enter burst time of P1: 5
Enter burst time of P2: 3
Enter burst time of P3: 2

Process BT WT TAT
P1 5 0 5
P2 3 5 8
P3 2 8 10
```

Q16. Write a C program to implement SJF CPU scheduling.

Answer:

```
#include <stdio.h>

int main()
{
    int n, i, j, temp;
    int bt[20], p[20], wt[20], tat[20];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        p[i] = i + 1;
        printf("Enter burst time of P%d: ", p[i]);
        scanf("%d", &bt[i]);
    }

    for(i = 0; i < n-1; i++)
        for(j = i+1; j < n; j++)
```

```
            if(bt[i] > bt[j])
            {
                temp = bt[i]; bt[i] = bt[j]; bt[j] = temp;
                temp = p[i]; p[i] = p[j]; p[j] = temp;
            }

    wt[0] = 0;
    for(i = 1; i < n; i++)
        wt[i] = wt[i-1] + bt[i-1];

    printf("\nProcess\tBT\tWT\tTAT\n");
    for(i = 0; i < n; i++)
    {
        tat[i] = wt[i] + bt[i];
        printf("P%d\t%d\t%d\t%d\n", p[i], bt[i], wt[i], tat[i]);
    }

    return 0;
}
```

Expected Output:

```
Enter number of processes: 3
Enter burst time of P1: 6
Enter burst time of P2: 2
Enter burst time of P3: 4

Process BT WT TAT
P2 2 0 2
P3 4 2 6
P1 6 6 12
```

Q17. Write a C program to implement Round Robin CPU scheduling.

Answer:

```
#include <stdio.h>

int main()
{
    int n, tq, i, time = 0, done;
    int bt[20], rem[20], wt[20], tat[20];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++)
    {
        printf("Enter burst time of P%d: ", i+1);
        scanf("%d", &bt[i]);
        rem[i] = bt[i];
    }

    printf("Enter time quantum: ");
    scanf("%d", &tq);

    do
    {
        done = 1;
        for(i = 0; i < n; i++)
        {
            if(rem[i] > 0)
            {
                done = 0;
                if(rem[i] > tq)
                {
                    time += tq;
                    rem[i] -= tq;
                }
                else
                {
                    time += rem[i];
                    wt[i] = time - bt[i];
                    rem[i] = 0;
                }
```

```
            }
        }
    } while(!done);

    for(i = 0; i < n; i++)
        tat[i] = bt[i] + wt[i];

    printf("\nProcess\tBT\tWT\tTAT\n");
    for(i = 0; i < n; i++)
        printf("P%d\t%d\t%d\t%d\n", i+1, bt[i], wt[i], tat[i]);

    return 0;
}
```

Expected Output:

```
Enter number of processes: 3
Enter burst time of P1: 5
Enter burst time of P2: 3
Enter burst time of P3: 2
Enter time quantum: 2

Process BT WT TAT
P1 5 4 9
P2 3 4 7
P3 2 4 6
```

4. Multithreading and Resource Utilization

Q18. Write a C program to create a thread using POSIX pthread.

Answer:

```
#include <stdio.h>
#include <pthread.h>

void *display(void *arg)
{
    printf("Thread is running\n");
    return NULL;
}

int main()
{
    pthread_t thread;

    pthread_create(&thread, NULL, display, NULL);
    pthread_join(thread, NULL);

    printf("Main thread completed\n");
    return 0;
}
```

Expected Output:

```
Thread is running
Main thread completed
```

Q19. Write a C program to create multiple threads and display their thread IDs.

Answer:

```
#include <stdio.h>
#include <pthread.h>

void *task(void *arg)
{
    printf("Thread ID = %lu\n", pthread_self());
    return NULL;
}

int main()
{
    pthread_t t1, t2, t3;
```

```
    pthread_create(&t1, NULL, task, NULL);
    pthread_create(&t2, NULL, task, NULL);
    pthread_create(&t3, NULL, task, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);

    printf("All threads completed\n");
    return 0;
}
```

Expected Output:

```
Thread ID = 1401
Thread ID = 1402
Thread ID = 1403
All threads completed
```

5. Inter-Process Communication and Shared Resource Management

Q20. Write a C program to demonstrate IPC using a pipe between parent and child.

Answer:

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>

int main()
{
    int fd[2];
    char message[] = "Hello from Parent";
    char buffer[100];

    pipe(fd);

    if (fork() == 0)
    {
        close(fd[1]);
        read(fd[0], buffer, sizeof(buffer));
        printf("Child received: %s\n", buffer);
    }
    else
    {
        close(fd[0]);
        write(fd[1], message, strlen(message) + 1);
    }

    return 0;
}
```

Expected Output:

```
Child received: Hello from Parent
```

Q21. Write a C program to demonstrate shared memory concept.

Answer:

```
#include <stdio.h>
#include <sys/shm.h>
#include <string.h>

int main()
{
    int shmid;
    char *str;

    shmid = shmget(IPC_PRIVATE, 100, IPC_CREAT | 0666);
    str = (char *)shmat(shmid, NULL, 0);

    strcpy(str, "Hello using Shared Memory");
    printf("Data in shared memory: %s\n", str);
```

```
    shmdt(str);
    shmctl(shmid, IPC_RMID, NULL);

    return 0;
}
```

Expected Output:

```
Data in shared memory: Hello using Shared Memory
```

Q22. Write a C program to demonstrate message queue communication.

Answer:

```
#include <stdio.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <string.h>

struct message
{
    long type;
    char text[100];
};

int main()
{
    int id;
    struct message msg;

    id = msgget(IPC_PRIVATE, 0666 | IPC_CREAT);

    msg.type = 1;
    strcpy(msg.text, "Hello using Message Queue");

    msgsnd(id, &msg, sizeof(msg.text), 0);
    msgrcv(id, &msg, sizeof(msg.text), 1, 0);

    printf("Received: %s\n", msg.text);

    msgctl(id, IPC_RMID, NULL);

    return 0;
}
```

Expected Output:

```
Received: Hello using Message Queue
```

6. Synchronization Mechanisms for Concurrent Systems

Q23. Write a C program using a mutex lock to protect a shared variable.

Answer:

```
#include <stdio.h>
#include <pthread.h>

int counter = 0;
pthread_mutex_t lock;

void *increment(void *arg)
{
    for(int i = 0; i < 1000; i++)
    {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main()
{
    pthread_t t1, t2;
```

```
    pthread_mutex_init(&lock, NULL);

    pthread_create(&t1, NULL, increment, NULL);
    pthread_create(&t2, NULL, increment, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("Counter = %d\n", counter);

    pthread_mutex_destroy(&lock);

    return 0;
}
```

Expected Output:

```
Counter = 2000
```

Q24. Write a C program to demonstrate the Producer-Consumer problem using semaphores.

Answer:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

int buffer = 0;
sem_t empty, full;

void *producer(void *arg)
{
    sem_wait(&empty);
    buffer = 10;
    printf("Produced: %d\n", buffer);
    sem_post(&full);
    return NULL;
}

void *consumer(void *arg)
{
    sem_wait(&full);
    printf("Consumed: %d\n", buffer);
    sem_post(&empty);
    return NULL;
}

int main()
{
    pthread_t p, c;

    sem_init(&empty, 0, 1);
    sem_init(&full, 0, 0);

    pthread_create(&p, NULL, producer, NULL);
    pthread_create(&c, NULL, consumer, NULL);

    pthread_join(p, NULL);
    pthread_join(c, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);

    return 0;
}
```

Expected Output:

```
Produced: 10
Consumed: 10
```

7. Deadlock Detection, Avoidance and Recovery

Q25. Write a C program to implement the Banker's Algorithm for deadlock avoidance.

Answer:

```
#include <stdio.h>

int main()
{
    int n, m, i, j, k;
    int alloc[10][10], max[10][10], avail[10];
    int need[10][10], finish[10] = {0}, safe[10];
    int count = 0, found;

    printf("Enter processes and resources: ");
    scanf("%d%d", &n, &m);

    printf("Enter Allocation Matrix:\n");
    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            scanf("%d",&alloc[i][j]);

    printf("Enter Max Matrix:\n");
    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            scanf("%d",&max[i][j]);

    printf("Enter Available Resources:\n");
    for(j=0;j<m;j++)
        scanf("%d",&avail[j]);

    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            need[i][j] = max[i][j] - alloc[i][j];

    while(count < n)
    {
        found = 0;

        for(i=0;i<n;i++)
        {
            if(!finish[i])
            {
                int possible = 1;

                for(j=0;j<m;j++)
                    if(need[i][j] > avail[j])
                        possible = 0;

                if(possible)
                {
                    for(k=0;k<m;k++)
                        avail[k] += alloc[i][k];

                    safe[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }

        if(!found)
            break;
    }

    if(count == n)
    {
        printf("System is in Safe State\n");
        printf("Safe Sequence: ");

        for(i=0;i<n;i++)
            printf("P%d ", safe[i]);
    }
    else
        printf("System is not in Safe State\n");

    return 0;
```

```
}
```

Expected Output:

```
System is in Safe State
Safe Sequence: P1 P3 P4 P0 P2
```

8. Memory Management and Page Replacement Strategies

Q26. Write a C program to implement FIFO page replacement.

Answer:

```
#include <stdio.h>

int main()
{
    int pages[20], frames[10];
    int n, f, i, j, k = 0, faults = 0, hit;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter reference string: ");
    for(i=0;i<n;i++)
        scanf("%d",&pages[i]);

    printf("Enter number of frames: ");
    scanf("%d",&f);

    for(i=0;i<f;i++)
        frames[i] = -1;

    for(i=0;i<n;i++)
    {
        hit = 0;

        for(j=0;j<f;j++)
            if(frames[j] == pages[i])
                hit = 1;

        if(!hit)
        {
            frames[k] = pages[i];
            k = (k + 1) % f;
            faults++;
        }
    }

    printf("Page Faults = %d\n", faults);

    return 0;
}
```

Expected Output:

```
Page Faults = 9
```

Q27. Write a C program to implement LRU page replacement.

Answer:

```
#include <stdio.h>

int main()
{
    int pages[20], frames[10], time[10];
    int n, f, i, j, pos, faults = 0, counter = 0;
    int hit, least;

    printf("Enter number of pages: ");
    scanf("%d", &n);

    printf("Enter reference string: ");
    for(i=0;i<n;i++)
```

```
        scanf("%d",&pages[i]);

    printf("Enter number of frames: ");
    scanf("%d",&f);

    for(i=0;i<f;i++)
    {
        frames[i] = -1;
        time[i] = 0;
    }

    for(i=0;i<n;i++)
    {
        hit = 0;
        counter++;

        for(j=0;j<f;j++)
        {
            if(frames[j] == pages[i])
            {
                hit = 1;
                time[j] = counter;
                break;
            }
        }

        if(!hit)
        {
            pos = -1;

            for(j=0;j<f;j++)
                if(frames[j] == -1)
                {
                    pos = j;
                    break;
                }

            if(pos == -1)
            {
                least = time[0];
                pos = 0;

                for(j=1;j<f;j++)
                    if(time[j] < least)
                    {
                        least = time[j];
                        pos = j;
                    }
            }

            frames[pos] = pages[i];
            time[pos] = counter;
            faults++;
        }
    }

    printf("Page Faults = %d\n", faults);

    return 0;
}
```

Expected Output:

```
Page Faults = 8
```

Q28. Write a C program to demonstrate paging address translation.

Answer:

```
#include <stdio.h>

int main()
{
    int page, offset, frame;
    int page_size = 100;
```

```
    printf("Enter page number: ");
    scanf("%d", &page);

    printf("Enter offset: ");
    scanf("%d", &offset);

    printf("Enter frame number: ");
    scanf("%d", &frame);

    int physical = frame * page_size + offset;

    printf("Physical Address = %d\n", physical);

    return 0;
}
```

Expected Output:

```
Enter page number: 2
Enter offset: 20
Enter frame number: 5
Physical Address = 520
```

9. File System and Disk Scheduling Analysis

Q29. Write a C program to implement FCFS disk scheduling.

Answer:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int n, i, head, total = 0, request[20];

    printf("Enter number of requests: ");
    scanf("%d", &n);

    printf("Enter request queue: ");
    for(i=0;i<n;i++)
        scanf("%d",&request[i]);

    printf("Enter initial head position: ");
    scanf("%d",&head);

    for(i=0;i<n;i++)
    {
        total += abs(request[i] - head);
        head = request[i];
    }

    printf("Total Head Movement = %d\n", total);

    return 0;
}
```

Expected Output:

```
Enter number of requests: 5
Enter request queue: 98 183 37 122 14
Enter initial head position: 53
Total Head Movement = 640
```

Q30. Write a C program to implement SSTF disk scheduling.

Answer:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int request[20], visited[20] = {0};
    int n, head, total = 0;
```

```
    int i, count, index, distance, min;

    printf("Enter number of requests: ");
    scanf("%d", &n);

    printf("Enter request queue: ");
    for(i=0;i<n;i++)
        scanf("%d",&request[i]);

    printf("Enter initial head position: ");
    scanf("%d",&head);

    for(count=0; count<n; count++)
    {
        min = 9999;
        index = -1;

        for(i=0;i<n;i++)
        {
            if(!visited[i])
            {
                distance = abs(request[i] - head);

                if(distance < min)
                {
                    min = distance;
                    index = i;
                }
            }
        }

        visited[index] = 1;
        total += min;
        head = request[index];
    }

    printf("Total Head Movement = %d\n", total);

    return 0;
}
```

Expected Output:

```
Total Head Movement = 236
```

Q31. Write a C program to implement SCAN disk scheduling.

Answer:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int req[20], n, head, disk_size;
    int i, j, temp, total = 0;

    printf("Enter number of requests: ");
    scanf("%d", &n);

    printf("Enter requests: ");
    for(i=0;i<n;i++)
        scanf("%d",&req[i]);

    printf("Enter initial head position: ");
    scanf("%d",&head);

    printf("Enter disk size: ");
    scanf("%d",&disk_size);

    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(req[i] > req[j])
            {
                temp=req[i];
```

```
                req[i]=req[j];
                req[j]=temp;
            }

    printf("SCAN order: ");

    for(i=0;i<n;i++)
        if(req[i] >= head)
        {
            printf("%d ", req[i]);
            total += abs(req[i]-head);
            head=req[i];
        }

    total += abs((disk_size-1)-head);
    head=disk_size-1;

    for(i=n-1;i>=0;i--)
        if(req[i] < head)
        {
            printf("%d ", req[i]);
            total += abs(req[i]-head);
            head=req[i];
        }

    printf("\nTotal Head Movement = %d\n", total);

    return 0;
}
```

Expected Output:

```
SCAN order: 98 122 183 37 14
Total Head Movement = 331
```

Q32. Write a C program to implement C-SCAN disk scheduling.

Answer:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int req[20], n, head, disk_size;
    int i, j, temp, total = 0;

    printf("Enter number of requests: ");
    scanf("%d",&n);

    printf("Enter requests: ");
    for(i=0;i<n;i++)
        scanf("%d",&req[i]);

    printf("Enter initial head position: ");
    scanf("%d",&head);

    printf("Enter disk size: ");
    scanf("%d",&disk_size);

    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(req[i] > req[j])
            {
                temp=req[i];
                req[i]=req[j];
                req[j]=temp;
            }

    for(i=0;i<n;i++)
        if(req[i] >= head)
        {
            total += abs(req[i]-head);
            head=req[i];
        }
```

```
    total += (disk_size-1)-head;
    total += disk_size-1;

    head=0;

    for(i=0;i<n;i++)
        if(req[i] < head)
            continue;

    printf("Total Head Movement = %d\n", total);

    return 0;
}
```

Expected Output:

```
Total Head Movement = calculated according to the given request queue and disk size.
```

10. Containerization and Resource Monitoring for AI/ML Workloads

Q33. Write Docker commands to pull an image, create a container, and run it.

Answer:

```
docker pull python:3.11
docker run -it python:3.11
docker ps
```

Expected Output:

```
Python container starts successfully and the running container is displayed by docker ps.
```

Q34. Write a Dockerfile for a simple Python application.

Answer:

```
FROM python:3.11

WORKDIR /app

COPY app.py .

CMD ["python", "app.py"]
```

Expected Output:

```
Docker image builds successfully and runs app.py inside the container.
```

Q35. Write Docker commands to build and run an image.

Answer:

```
docker build -t my-python-app .
docker run my-python-app
```

Expected Output:

```
Successfully built image: my-python-app
Application runs inside the container.
```

Q36. Write Linux/Docker commands to monitor CPU, memory and process utilization of a container.

Answer:

```
docker ps
docker stats
docker top <container_id>
ps aux
free -h
df -h
```

Expected Output:

```
CONTAINER ID   CPU %   MEM USAGE / LIMIT
123abc         2.5%    120MiB / 1GiB
```

Note: Process IDs, thread IDs, system statistics, and some algorithm outputs can vary depending on the system and input values.